

KNN Projekt

Adam Malysák – xmalys02, Ondřej Lukášek – xlukas15, Patrik Nehéz – xnehez01

Květen 2026

1 Cíl projektu

Cílem projektu je adaptace VLLM pro extrakci obsahů knih a otestovat jejich schopnost určit ne jenom samotný text údajů, ale i jejich geometrickou pozici a produkci strukturovaného výstupu v konkrétním formátu. A poté hlavně adaptovat menší VLLM tak, aby byly schopny extrahovat obsahy včetně geometrické pozice.

K dispozici máme obrázky, tedy naskenované obsahy knih, jejich anotace a OCR výstupy. Bylo potřeba ke každému obrázku vytvořit ground truth *json* dokument (vytvořit ground truth dataset). Dále vyzkoušet jak si modely vedou samy o sobě, a poté je dotrénovat pomocí **LoRA** a vyhodnotit jejich funkcionalitu. Modely, se kterými jsme nakonec pracovali jsou následující: k dotrénování „menší“ modely **Qwen3-VL-8B-Instruct** [6], **Qwen3-VL-4B-Instruct** [5], **Qwen3-VL-2B-Instruct** [4]. Pro srovnání jak si vedou větší modely **GPT-4.5** [3], **GPT-4o** [2] a původně bylo v plánu dotrénovat i **Florence-2** [7], ale nakonec jsme jen vyzkoušeli jak si vede bez finetuningu. Veškerá práce s Qweny probíhala na MetaCentru.

2 Dataset

Dataset tvoří obrázky a k nim korespondující *json* dokumenty, které mají následující strukturu definovanou v Pydantic:

```
1 from typing import Optional, List, Tuple
2 from pydantic import BaseModel
3
4 class Chapter(BaseModel):
5     name: Optional[str] = None
6     chapter_number: Optional[str] = None
7     page_number: Optional[str] = None
8     description: Optional[str] = None
9
10    name_bbox: Optional[Tuple[Tuple[int, int], Tuple[int, int]]] = None
11    chapter_number_bbox: Optional[Tuple[Tuple[int, int], Tuple[int, int]]] = None
12    page_number_bbox: Optional[Tuple[Tuple[int, int], Tuple[int, int]]] = None
13    description_bbox: Optional[Tuple[Tuple[int, int], Tuple[int, int]]] = None
14
15    subchapters: Optional[List['Chapter']] = []
```

Listing 1: Definice datového modelu v Pythonu (Pydantic)

Informace k extrakci tedy jsou: **název kapitoly**, **číslo kapitoly**, **číslo strany kapitoly**, **podnadsis kapitoly**, **názvy podkapitol** (bereme v úvahu jen jednu úroveň podkapitol) a **bounding boxy** všech zmíněných údajů.

K dispozici byly pouze samotné anotace obrázků (obsahů knih) a OCR výstupy. Ty bylo potřeba na sebe namapovat a vytvořit pro každý obrázek jeho ground truth výstup (*json*).

2.1 Vizualizace

K ladění skriptu na mapování OCR a anotací jsme vytvořili vizualizační skript, který do obrázku vykreslí obsah jsonu. Skript taky zvýrazňoval bounding boxy, do kterých se nepodařilo z OCR namapovat žádný text, což nám později usnadnilo nějaké mírné manuální úpravy datasetu. Příklad vizualizace lze vidět na obrázku 1.

2.2 Ground truth dataset

Ground truth dataset byl tedy vytvořen namapováním anotací a OCR pomocí odladěného mapovacího skriptu. Celkem ho tvoří 1426 obrázků a k nim odpovídající jsony. Kvalita datasetu je diskutabilní díky přesnosti OCR i samotných

O B S A H .	
	Str.
Účel ranného chovu drůbeže	3
Výběr vajec k línutí	4
Úprava hnízda	5
Línutí drůbeže	9
Opatrování vyřídě drůbeže	13
a) Kuřata	14
b) Housata	17
c) Káčata	19
d) Krůtata	20
e) Perličky	21
f) Pávi	22
g) Labutě	23
Jakou drůbež máme chovati?	24
Slepice	25
1. Slepice pro vejce	25
2. Slepice pro vejce a chutné maso	29
3. Slepice pro všechno i odchov kuřat	33
Husy	37
Kachny	39
Krůty	43
Perličky	45
Pávi	47

Obrázek 1: Příklad vizualizace ground truth jsonu na odpovídajícím obrázku (obsahu).

anotací. A to i přes manuální opravu některých chyb, které zvýraznil vizualizační skript. Dále původní dataset obsahoval obrázky, na kterých nebyly vůbec obsahy knih. Takové obrázky jsme vyfiltrovali. Po těchto úpravách zbylo právě zmíněných 1426 obrázků a k nim 1426 ground truth jsonů.

Ještě stojí za zmínku několik vlastností obsahů, které modelům dělají problémy. Obsahy ground truth jsonů jsou seřazeny podle kapitol. Občas některý obsah nezačíná „na začátku“, ale je to třeba několikátá strana obsahu. To by nebyl problém, až na to, že takové obsahy nemusí začínat kapitolou, ale zanořenou kapitolou. Dále můžou mít dvousloupcové, nebo dokonce i třísłoupcové rozložení stránky. A za zmínění ještě stojí fakt, že jedna z informací, kterou chceme extrahovat, se vyskytuje velmi málo, a modely ji pravděpodobně proto těžko zpracovávají (řeč je o podnadpisech).

3 Modely a trénování

Jak již bylo zmíněno v úvodu, pracovali jsme s modely: **Qwen3-VL-8B-Instruct** [6], **Qwen3-VL-4B-Instruct** [5], **Qwen3-VL-2B-Instruct** [4], **GPT-4.5** [3], **GPT-4o** [2]. S tím, že Qwen modely jsme i dotrénovávali, a GPT modely jsme jen zkoušeli abychom věděli jak si s takovou úlohou zvládne poradit velký VLLM.

3.1 GPT

Co se GPT modelů týče, tak kromě zmíněných modelů jsme zkoušeli i jejich *mini* varianty, ale moc dobře to nezvládaly, takže pro vyhodnocení na batchi o více datech jsme použili pouze již zmíněné verze. Je doporučeno se souřadnicemi pracovat v rozsahu od 0 do 1000, takže výstupní souřadnice bounding boxů byly v tomto rozsahu a aby šli potom při evaluaci porovnat, převedli jsme je zpět do absolutních souřadnic. Výsledky jsou popsány v sekci 5.

3.2 Qwen

Jelikož cílem bylo adaptovat menší modely, dotrénovávali jsme 3 různě velké Qwen modely: 8, 4 a 2 miliardy parametrů. Použili jsme LoRA finetuning (podle návodu: [1]).

Pokud je obrázek moc velký, model si ho sám zmenšuje. Při trénování jsme obrázky zmenšili sami a upravili jsme hodnoty souřadnic bounding boxů v ground truth jsonech, aby výstup modelu mohl odpovídat ground truth. Souřadnice na výstupu z modelu byly v rozsahu 0 až 1000, takže jsme stejným způsobem upravili i souřadnice v ground truth. Obrázky jsme zmenšili tak, aby delší strana měla maximálně 1024 pixelů, a Qwen je nemusel zmenšovat sám.

Data jsme rozdělili v poměru 9:1, 9 na trénování a 1 na testování. Dvoumiliardový Qwen jsme pomocí LoRA trénovali 20 epoch. Po třech epochách už loss oscilovala a model už se nezlepšoval. Čtyř a osmimiliardový Qwen už jsme proto trénovali jen 5 epoch. Výsledky jsou popsány v sekci 5.

4 Hodnocení výstupů

Vyhodnocení modelů je prováděno pomocí evaluačního skriptu, který vezme složku s jsony vygenerované modelem a složku s jejich odpovídajícími ground truth jsony a vypočítá z nich různé metriky. Metriky týkající se schopnosti modelu číst text, schopnosti udávat správné geometrické pozice (bounding boxy) údajů a schopnosti dodržovat správnou strukturu jsonu. Všechny metriky se počítají pro každý údaj (název kapitoly, číslo kapitoly, ...) zvlášť.

Pokud json není validní nebo nedodržuje pydantic definovanou strukturu, skript z něj nic nevyhodnocuje a přeskočí ho. Z tohoto důvodu je na výstupu kromě výše zmíněných metrik i informace o tom, kolik bylo zpracováno jsonů a procento validních souborů. Občas se stalo, že výstupní json byl moc velký a nevyšly na něj tokeny. V takovém případě není dokončený a tím pádem není vůbec validní. Přeskočených jsonů byla přeskočena právě z tohoto důvodu (jinak sktrukturu dodržely).

K vyhodnocení schopnosti číst text používáme metriku **character error rate (CER)**. Ta je vypočítána pro každý údaj (název kapitoly, číslo kapitoly, ...) zvlášť. A to jednoduše z jednoho stringu, ve kterém jsou všechny texty těchto údajů pospojovány a odděleny mezerou.

K vyhodnocení schopnosti produkovat správné bounding boxy údajů používáme, opět pro každý údaj zvlášť, 2 metriky, a to **f1 skóre** a **průměrné IoU**. Tady zatím neřešíme, zda jsou údaje strukturálně správně (například zda je číslo kapitoly přiřazeno správné kapitole), ale čistě jen vezmeme všechny bounding boxy jedné kategorie (jednoho údaje, například čísel kapitol) a porovnáváme je. F1 skóre nám zjednodušeně říká, jak dobře se modelu daří nacházet správné bounding boxy. Vypočítá se překryv (IoU) a pokud přesahuje nějaký práh, bereme ho jako *True positive*. Z takových bounding boxů pak počítáme průměrný překryv (Mean IoU). Pomocí počtu True positive bounding boxů, počtu modelem predikovaných bounding boxů a počtu ground truth bounding boxů daného údaje vypočítáme počet *False positive* a *False negative* bounding boxů. Z těchto údajů pak vypočítáme f1 skóre:

$$precision = tp / (tp + fp) \quad recall = tp / (tp + fn)$$

$$f1_score = 2 * (precision * recall) / (precision + recall)$$

Vyhodnocení toho, jak strukturálně správně model zvládá generovat výstup dost závisí na tom, jak zvládá text a jak zvládá bounding boxy, protože k tomu abychom řekli, že model správně přiřadil například číslo kapitoly správné kapitole záleží i na tom, jak zvládl její text a bounding box. Párování údajů probíhá podle textu a bounding boxů, s tím že jsou tolerovány mírné odchylky jak v textu, tak v bounding boxech. Skript jde po kapitolách a snaží napasovat kapitoly z výstupu modelu na ground truth kapitoly, a pokud se to podaří, tak i na podkapitoly daných kapitol. Pro každou správně napasovanou kapitolu se snaží ověřit, zda souhlasí i ostatní údaje (jako například číslo kapitoly) a drží si počty namapovaných údajů. Z těchto údajů pak vypočítá f1 skóre pro kapitoly (a podkapitoly), které nám vlastně říká jak moc se výsledný strom kapitol a podkapitol z modelu shoduje se stromem v tround truth. Ze spárovaných (pod)kapitol se pak ještě vypočítá přesnost párování ostatních údajů, tedy čísel kapitol, čísel stran a podnadpisů kapitol.

Příklad evaluačního výstupu:

```
Directories: openai/out_trim_normalized openai/out_4o/
Number of files: 1391
Valid files ratio: 0.9985621854780733
#####
#### TEXT ####
Names CER: 0.16614254664948055
Chapter numbers CER: 0.23947631278538814
Page numbers CER: 0.1099046125702339
Descriptions CER: 0.951052899000734

#### BBOXES ####
Names bbox f1_score: 0.002348723654119538
Names bbox mean iou: 0.7919955253601074
Chapter numbers bbox f1_score: 0.0005736576411197799
Chapter numbers bbox mean iou: 0.7819338440895081
Page numbers bbox f1_score: 0.00044504773136918934
```

```
Page numbers bbox mean iou: 0.7877779006958008
Descriptions bbox f1_score: 0.002314814814814815
Descriptions bbox mean iou: 0.7763023376464844
```

```
#### JSON STRUCTURE ####
```

```
Structure chapters f1_score: 0.00867705100780332
Structure page number accuracy: 0.4789525070357204
Structure chap number accuracy: 0.5619726975953898
Structure description accuracy: 0.0
```

5 Výsledky

V této sekci vyhodnotíme a porovnáme modely, se kterými jsme pracovali.

5.1 GPT

S GPT modely jsme pracovali abychom zjistili, jak si s touto úlohou poradí nějaký novější VLLM. Jelikož jsme nemuseli nic dotrénovávat, mohli jsme nechat modely zpracovat více dat. GPT-4o jsme nechali zpracovat 1391 naskenovaných obsahů a GPT-5.4 pouze 399. Důvodem bylo vyčerpání rozpočtu. Tento nepoměr je třeba brát v potaz při analýze výsledků. Výsledky jsou vizualizovány v grafech 2. U obou modelů bylo validních přibližně 99.8% výstupů (jsonů), takže formát zvládal generovat syntakticky správně.

Z grafů lze vyčíst, že co se textu týče, chybovost (CER) u všech kategorií kromě podnadpisů se vesměs drží pod 0.2. S podnadpisy mají modely problémy. Dále musíme znovu připomenout, kvalita ground truth datasetu (výstup OCR použitého pro vytvoření datasetu) není stoprocentní, což chybovost pravděpodobně ještě zvyšuje.

Co se bounding boxů týče, GPT-5.4 je o něco lepší, než GPT-4o, zejména co se názvů (pod)kapitol týče. Ale hodnoty f1_score jsou u obou modelů stále velmi nízko. Modelům se nacházet správné bounding boxy moc nedaří.

A co se týče strukturálních metrik, f1_score je velmi nízko, takže stromová struktura kapitol a podkapitol se vesměs neshoduje. A pokud už se shoduje, tak přesnosti určování čísel stran a čísel kapitol se dostávají maximálně na 0.7. GPT-5.4 je opět o něco lepší, než GPT-4o.

Na závěr by se dalo konstatovat, že ani jeden z modelů úlohu na přijatelné úrovni nezvládl.

5.2 Qwen

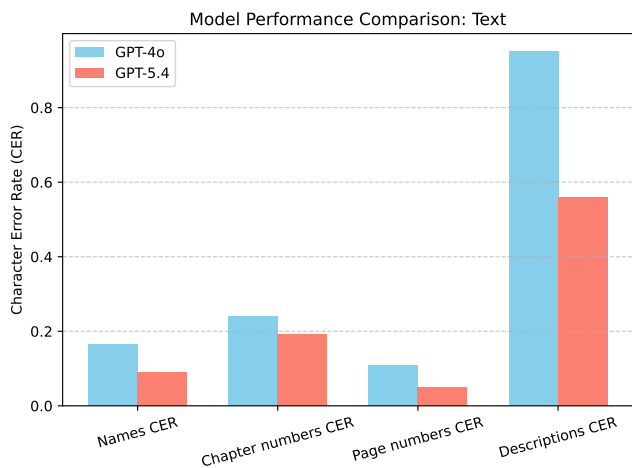
S Qwen modely jsme pracovali tak, že jsme dotrénovali pomocí LoRA (návod: [1]) a poté jsme je otestovali na vzorku dat. Před samotným dotrénováním jsme udělali pár experimentů s 8B modelem, a největší problém byl, že se modelu nedařilo produkovat výstup ve správném formátu. Náš evaluační skript by všechny jsony přeskočil a žádné metriky by nevypočítal. Jinak ale dle namátkové kontroly zvládal jak pozice (ověřeno vizualizací), tak text. Výsledky a porovnání modelů po dotrénování jsou vizualizovány v grafech 3.

Z grafů lze vyčíst, že co se textu týče, největší problém je opět s podnadpisy. U podnadpisů jsou také znatelnější rozdíly mezi modely, u ostatních údajů je chybovost (CER) textu napříč modely podobná. U názvů se pohybuje kolem 0.15, u čísel kapitol kolem 0.25 a u čísel stran kolem 0.1. Nejmenší chybovost u všech údajů má Qwen 4B, vypadá to tedy, že s textem si poradil nejlépe, i když, jak bylo zmíněno, rozdíly nejsou velké. Qweny si vedou podobně, jako GPT modely.

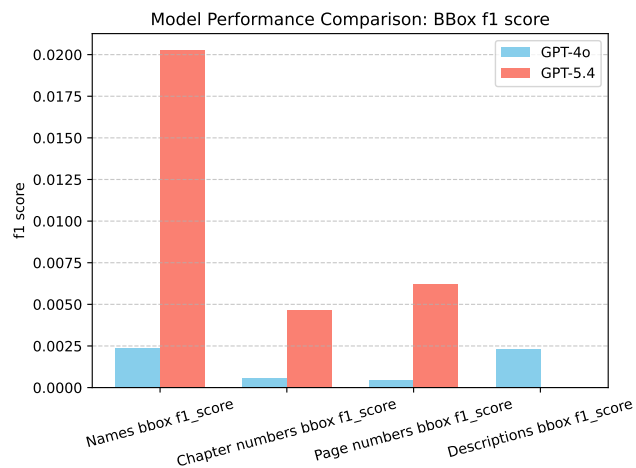
Když se podíváme na bounding boxy, vedou si dotrénované Qweny mnohem lépe, než GPT modely. Nejlépe si v každé kategorii, kromě podnadpisů, vede 8B. U podnadpisů je to 4B. Stejně jako u GPT nejvyšší f1_score jsou u názvů (pod)kapitol a pro ostatní údaje je f1_score nižší.

Strukturální metriky jsou také na úplně jiné úrovni, než u GPT. Hlavně nejdůležitější **Structure chapters f1_score**. U všech tří modelů je nad 0.7, což je oproti GPT obrovský rozdíl. U správně spárovaných (pod)kapitol se přesnost určování čísel stran a čísel kapitol pohybuje kolem 0.8. U podnadpisů značně vede Qwen 4B. Ale jak bylo již zmíněno, podnadpisů je málo a není to nejobecnější míra kvality.

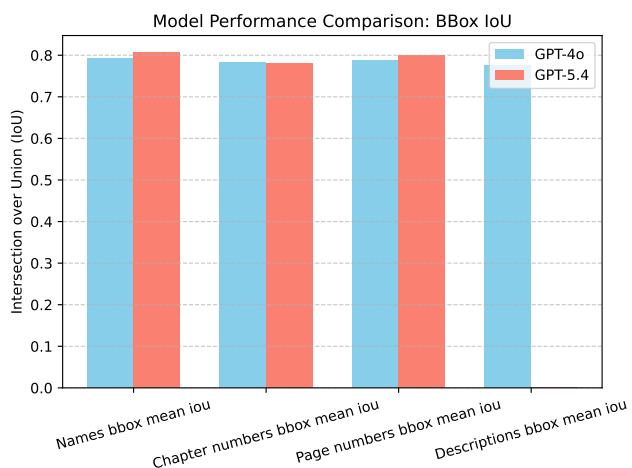
Nejlépe si obecně vede osmimiliardový Qwen. Soudě hlavně podle bounding boxů a zpracování struktury. Qwen 4B nezaostával, a dokonce si vedl lépe u podnadpisů. Qwen 2B měl velký problém s podnadpisy, ale jinak za většími modely příliš nezaostával. Co se týče zpracování textu, byl nejlepší. Závěrem lze konstatovat, že dotrénované Qweny úlohu dle výsledků 3 zvládají vesměs dobře.



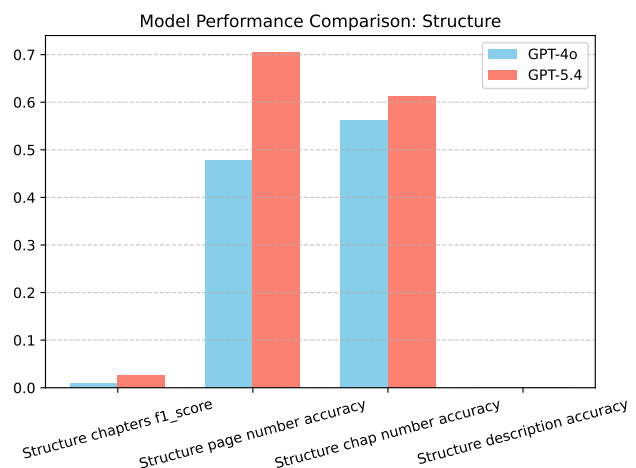
(a) Porovnání schopnosti modelů zpracovávat text z naskenovaných obsahů.



(b) Porovnání schopnosti modelů nacházet správné bounding boxy.

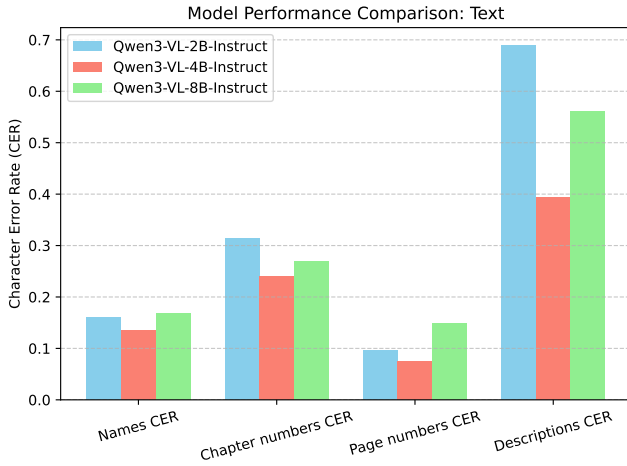


(c) Porovnání průměrných IoU u správně nalezených bounding boxů.

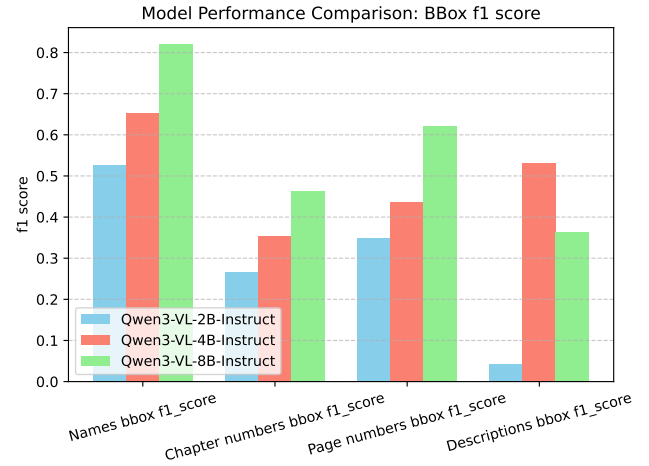


(d) Porovnání schopnosti modelů generovat správně strukturované výstupy.

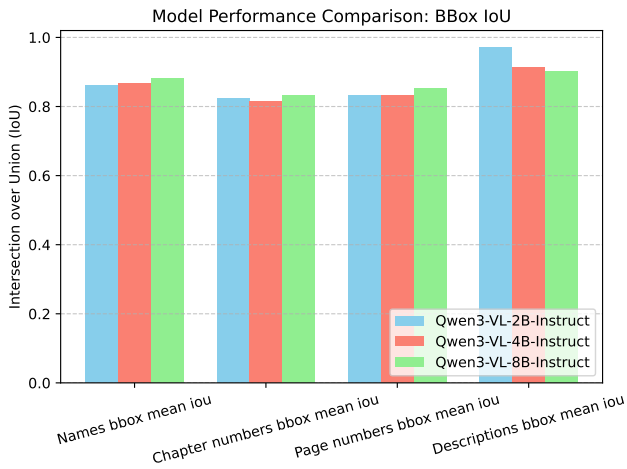
Obrázek 2: Porovnání GPT-4o a GPT-5.4. Všechny metriky rozděleny podle údajů, které zpracovávají. Názvy, čísla, čísla stran a podnadpisy (pod)kapitol



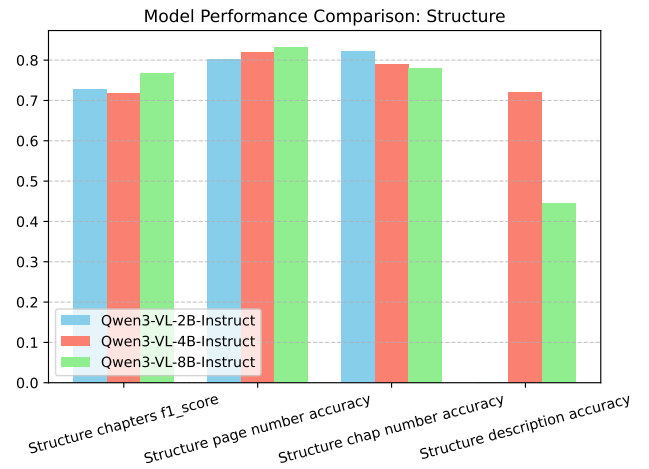
(a) Porovnání schopnosti modelů zpracovávat text z naskenovaných obsahů.



(b) Porovnání schopnosti modelů nacházet správné bounding boxy.



(c) Porovnání průměrných IoU u správně nalezených bounding boxů.



(d) Porovnání schopnosti modelů generovat správně strukturované výstupy.

Obrázek 3: Porovnání Qwenů 8B, 4B a 2B. Všechny metriky rozděleny podle údajů, které zpracovávají. Názvy, čísla, čísla stran a podnadpisy (pod)kapitol. Procenta validních výstupů jsou 92% pro 2B, 97% pro 4B a 98% pro 8B

Reference

- [1] DataCamp. Fine-tuning qwen3-vl 8b: A step-by-step guide, jan 2026. Dostupné z: <https://www.datacamp.com/tutorial/fine-tuning-qwen3-vl-8b>. URL: <https://www.datacamp.com/tutorial/fine-tuning-qwen3-vl-8b>.
- [2] OpenAI. Ahoj gpt-4o, may 2024. Dostupné z: <https://openai.com/cs-CZ/index/hello-gpt-4o/>. URL: <https://openai.com/cs-CZ/index/hello-gpt-4o/>.
- [3] OpenAI. Představujeme gpt-4.5, feb 2025. Dostupné z: <https://openai.com/cs-CZ/index/introducing-gpt-4-5/>. URL: <https://openai.com/cs-CZ/index/introducing-gpt-4-5/>.
- [4] Qwen Team. Qwen3-vl-2b-instruct, 2025. Dostupné z: <https://huggingface.co/Qwen/Qwen3-VL-2B-Instruct>. URL: <https://huggingface.co/Qwen/Qwen3-VL-2B-Instruct>.
- [5] Qwen Team. Qwen3-vl-4b-instruct, 2025. Dostupné z: <https://huggingface.co/Qwen/Qwen3-VL-4B-Instruct>. URL: <https://huggingface.co/Qwen/Qwen3-VL-4B-Instruct>.
- [6] Qwen Team. Qwen3-vl-8b-instruct, 2025. Dostupné z: <https://huggingface.co/Qwen/Qwen3-VL-8B-Instruct>. URL: <https://huggingface.co/Qwen/Qwen3-VL-8B-Instruct>.
- [7] Bin Xiao, Haiping Wu, Weijian Xu, Xiyang Dai, Houdong Hu, Yumao Lu, Michael Zeng, Ce Liu, and Lu Yuan. Florence-2: Advancing a unified representation for a variety of vision tasks, 2023. URL: <https://huggingface.co/papers/2311.06242>, arXiv:2311.06242.